Appendix C

Compilation and Execution in C Programming

[1] Breeha Qasim, [2]Bilal Wajid

[1] Habib University

Although it may appear that all it takes to compile and run a C program is one command, there is actually a complex series of steps involved that turn human-readable code into binaries that can be executed by a computer. While this procedure may not always be apparent to a beginner, it is crucial to comprehend if you wish to gain confidence in handling real-world programs. Usually, the compilation flow consists of the following steps: preprocessing, compilation, assembly, and linking. Each step has a specific function and is managed by tools like the linker (ld), the assembler (as), the preprocessor (cpp), and the compiler driver (gcc).

Thankfully, for most of the daily programming, the gcc command abstracts away these separate parts and provides a single interface. But as your programs get bigger and more complicated, you'll need to take charge of them by modifying compiler flags, adding more libraries, and improving speed. In this section, you will learn about these tools and practices, as well as how to compile, run, and comprehend the real-world effects of building a C program on Unix-based computers.

I. Compiling and Running a Program

Compiling and running your first C program comes after you've created it and saved it in a file (for example, `hello.c`). The procedures vary a little based on your operating system. Throughout, the GNU Compiler Collection, or `gcc`, will be our compiler.

On Ubuntu or WSL (Linux Shell)

Open your terminal and navigate to the directory containing `hello.c`. Then, compile the program using:

```
$ gcc hello.c
```

By default, this command will generate an executable file named `a.out`, which stands for "assembler output". This is the traditional name used by the UNIX toolchain if no output name is explicitly provided. You can verify this by listing the files in the directory after compilation.

To execute the file, run:

```
$ ./a.out
```

You should see:

```
hello, world
```

If you prefer a custom name for the executable, use the `-o` flag:

```
$ gcc -o hello hello.c
$ ./hello
```

On macOS

macOS also uses a terminal and supports `gcc` or Apple's `clang` compiler. The commands are the same:

```
gcc -o hello hello.c
./hello
```

The output should be:

```
hello, world
```

Note: On macOS, gcc may be a symbolic link to clang, which works similarly for basic C programs.

On Windows (MinGW Command Prompt)

If you installed GCC via MinGW-w64, open the MinGW command prompt or Command Prompt with environment variables set. Navigate to the directory where your `hello.c` file is saved. Compile using:

```
gcc hello.c -o hello.exe
```

Then run:

```
hello.exe
```

Expected output:

```
hello, world
```

In all cases, the compiler takes your source code and passes it through several stages: preprocessing, compiling, assembling, and linking. The result is an executable binary that your operating system can run.

Don't worry about these steps for now—`gcc` handles all of them automatically. Later sections will delve deeper into how this process works and how you can control it more precisely.

2. Understanding Compilation

The `gcc` command acts as a compiler driver — it doesn't do the compilation itself, but instead orchestrates several stages to produce an executable from source code:

- Preprocessing (`cpp`): Handles directives such as `#include` and `#define`, producing an expanded source file
- Compilation (`cc1`): Translates pre-processed C code into assembly code specific to the system's architecture.
- Assembly (`as`): Converts assembly instructions into machine-level object code.
- Linking (`ld`): Combines object files and links against required libraries to produce the final executable.

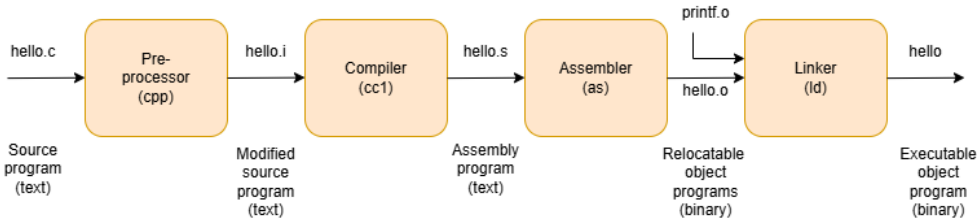

Figure 2.1: The compilation system

Fortunately, `gcc` manages all of these steps behind the scenes. By default, if you do not specify any flags, running:

```
gcc hello.c
```

will generate an executable file named `a.out`. However, this behaviour can be modified using command-line options (flags).

Useful GCC Flags

Here are some commonly used flags when compiling C programs:

```
gcc -o hello hello.c      # -o specifies the name of the output file
gcc -Wall hello.c         # -Wall enables all compiler warnings
gcc -g hello.c              # -g includes debug symbols for use with
gdb
```

```
gcc -O2 hello.c              # -O2 enables optimization for faster
performance
```

You can also combine these flags:

```
gcc -Wall -g -O2 -o hello hello.c
```

It is strongly advised that you utilize `-Wall` during development out of all of these. Fixing compiler warnings early can save hours of troubleshooting later because they are frequently the first indication of subtle defects or logical mistakes.